7-2 输入输出系统

- 1. 输入输出系统总览
  - I/O 接口的位置
    - **I/O 接口**：连接外设和主机的桥梁。
    - **主机侧**：通常称为内部接口，通过 I/O 总线与 CPU、内存相连。
    - **外设侧**：通常称为外部 I/O 接口，通过 USB 线、网线等连接外设。
    - **核心目标**：让 CPU、主存与各种速度不同、格式不同、控制方式不同的外设可靠通信。
  - I/O 接口的基本职能
    - **数据缓冲**：通过数据缓冲寄存器匹配主机和外设的工作速度。
    - **错误或状态检测**：通过状态寄存器保存设备状态和错误信息。
    - **控制和定时**：接收系统总线传来的控制命令和定时信号。
    - **数据格式转换**：在外部接口格式与内部接口格式之间转换。
- 2. I/O 接口的通用结构
  - 主机侧结构
    - **数据线**：在数据缓冲寄存器与内存或 CPU 寄存器之间传送数据。
    - **地址线**：给出要访问的 I/O 接口或接口内部寄存器地址。
    - **控制线**：传送读/写控制信号，也可包含仲裁信号、握手信号等。
    - **连接关系**：I/O 接口在主机侧通过 I/O 总线与 CPU、内存相连。
  - 接口内部寄存器
    - **数据缓冲寄存器**：暂存 CPU/主存与外设之间交换的数据。
    - **状态寄存器**：记录接口和设备的状态信息，并通过数据线送给 CPU 查询。
    - **控制寄存器**：接收 CPU 对外设的控制命令。
    - **状态/控制寄存器合并**：由于状态信息和控制命令传送方向相反，访问时间通常错开，有些接口会把二者合并。
  - 地址译码和 I/O 控制逻辑
    - **地址译码**：根据地址线选择要和主机交换数据的接口寄存器。
    - **读/写控制**：控制线上的读/写信号可能参与地址译码。
    - **命令译码**：对控制寄存器中的命令字进行译码。
    - **协调作用**：控制数据缓冲、状态收集、外设控制和数据传送。
  - 外设界面控制逻辑
    - **发送控制信号**：把控制命令经外设界面控制逻辑送给外设。
    - **收发数据**：把数据缓冲寄存器的数据发送给外设，或从外设接收数据到数据缓冲寄存器。
    - **收集状态**：把外设状态写入状态寄存器。
    - **本质**：把主机侧的总线信号转换为具体外设能理解的操作。
- 3. I/O 数据传送控制方式
  - 三种方式
    - **程序直接控制**：也称程序查询方式。
    - **中断控制**：也称程序中断 I/O 方式。
    - **DMA 控制**：直接存储器存取方式。
  - 三者核心区别

方式	谁主要控制	CPU 是否忙等	适用场景
程序查询	CPU 程序	常常忙等	简单、低速设备
中断控制	CPU + 中断机制	不必一直等待	随机、低速或中速 I/O
DMA 控制	DMA 控制器	传送过程基本不参与	磁盘等高速、批量数据传送

- 4. 程序直接控制方式
  - 基本思想
    - **方式**：CPU 通过程序定时或持续查询 I/O 接口状态。
    - **动作**：当外设准备好后，CPU 才执行一次数据传送。
    - **别名**：程序查询方式。
  - 基本流程
    - 1. **预置传送参数**。
    - 2. **启动外设**。
    - 3. **读取外设状态**。
    - 4. **判断外设是否准备就绪**。
    - 5. **传送一次数据**。
    - 6. **修改传送参数**。
    - 7. **判断是否传送完毕**。
    - 8. **未完成则继续查询，完成则结束**。
  - 优点
    - **简单易控制**：程序逻辑直观。
    - **接口控制逻辑少**：外围接口硬件较简单。
    - **实现成本低**：适合简单 I/O 场景。
  - 缺点
    - **CPU 忙等待**：外设未就绪时，CPU 反复读取接口状态。
    - **时间浪费严重**：CPU 速度远快于外设，大量时间耗在等待上。
    - **并行性差**：外设和 CPU 难以充分并行工作。
  - 查询方式细分
    - **独占查询**：设备启动后，CPU 一直查询接口状态，几乎 100% 时间用于 I/O。
    - **定时查询**：CPU 周期性查询接口状态，查询间隔与设备数据传输速率有关。
- 5. 中断控制方式
  - 基本思想
    - **别名**：程序中断 I/O 方式。
    - **并行工作**：外设和 CPU 可以并行工作。
    - **请求方式**：外设完成准备后向 CPU 发出中断请求。
    - **CPU 动作**：在可响应中断时，CPU 暂停当前程序，转去执行中断服务程序。
  - 中断 I/O 的优势
    - **减少忙等待**：CPU 不必一直查询外设状态。
    - **提高并行性**：外设工作时 CPU 可继续执行其他程序。
    - **响应更灵活**：外设需要服务时再通知 CPU。
  - 中断 I/O 的基本过程
    - 1. **CPU 启动外设**。
    - 2. **外设独立工作**。
    - 3. **外设完成或需要服务时发出中断请求**。
    - 4. **CPU 响应中断**。
    - 5. **CPU 执行中断服务程序**。
    - 6. **服务完成后返回原程序继续执行**。
- 6. 中断响应过程
  - 中断响应三步
    - **关中断**：暂时禁止新的中断响应。
    - **保存断点**：保存原程序下一步要执行的位置。
    - **引出中断服务程序**：找到中断服务程序入口地址并送入 PC。
  - 中断隐指令
    - **定义**：中断响应阶段由硬件自动完成的一系列操作。
    - **组成**：关中断、保存断点、引出中断服务程序。
    - **特点**：不是程序员显式写出的指令，而是硬件直接实现。
  - 为什么响应时要关中断
    - **原因**：保护断点和现场时不能被更高级中断打断。
    - **风险**：若保存过程被打断，可能导致断点或现场不完整，无法恢复原程序。
    - **结果**：中断响应过程中通常先关中断。
  - 断点保存
    - **中断断点**：通常是下一条指令的地址。
    - **异常断点**：异常指令通常没有执行成功，处理后要重新执行，因此断点常为当前指令地址。
    - **保存位置**：断点可保存在栈或特定寄存器中。
  - 引出中断服务程序
    - **中断向量**：中断服务程序入口地址，也可包含初始 PSW。
    - **装入 PC**：CPU 根据中断向量找到服务程序入口，并把入口地址送入 PC。
    - **完成标志**：CPU 开始执行中断服务程序时，中断响应完成。
- 7. 中断响应条件
  - 必要条件
    - **CPU 处于开中断状态**。
    - **至少有一个未被屏蔽的中断请求**。
    - **当前指令刚执行完**。
  - 补充理解
    - **非流水线处理器**：当前指令刚执行完时，PC 通常已保存下一条指令地址。
    - **被屏蔽中断**：即使有请求，若被屏蔽也不会被响应。
    - **响应时机**：通常在指令边界响应中断。
- 8. 中断源识别和判优
  - 软件查询方法
    - **基本方法**：检测到中断请求后，CPU 转到中断查询程序。
    - **查询顺序**：按优先级依次查询各设备是否有中断请求。
    - **跳转服务**：转到第一个查到有请求的设备对应中断服务程序。
    - **优点**：硬件结构简单，优先级可通过软件查询顺序调整。
    - **缺点**：软件查询速度慢，实时性较差。
  - 硬件判优方法
    - **基本方法**：通过中断控制器中的判优电路和编码器识别中断源。
    - **向量中断**：硬件给出中断源标识信息，CPU 据此找到中断服务程序入口。
    - **现代特点**：现代计算机大多采用独立请求方式进行中断源识别和判优。
    - **优点**：响应速度快，适合多个中断源并发请求。
  - 中断控制器基本结构
    - **中断请求寄存器**：保存各中断源请求状态。
    - **中断屏蔽字寄存器**：决定哪些中断请求被屏蔽。
    - **中断判优电路**：在未屏蔽请求中选择最高优先级。
    - **编码器**：输出中断类型号或中断源标识信息。
  - 中断向量表
    - **中断向量**：中断服务程序首地址和初始 PSW 等信息。
    - **中断向量表**：多个中断向量按表项组织。
    - **表项编号**：可通过中断类型号找到对应中断向量位置。
- 9. 中断处理过程
  - 单级中断
    - 1. **关中断**。
    - 2. **保存断点**。
    - 3. **中断服务程序寻址**。
    - 4. **保存现场**。
    - 5. **执行中断服务程序**。
    - 6. **恢复现场**。
    - 7. **开中断**。
    - 8. **中断返回**。
  - 中断响应与中断处理的区别
    - **中断响应**：硬件自动完成，主要包括关中断、保存断点、中断服务程序寻址。
    - **中断处理**：执行中断服务程序，包含保存现场、具体服务、恢复现场、中断返回等。
    - **记忆**：响应偏硬件，处理偏服务程序。
- 10. 多重中断
  - 基本概念
    - **多重中断**：处理一个中断时又出现新的中断请求，或同时出现多个中断请求。
    - **通常范围**：多重中断一般指可屏蔽中断。
    - **关键机制**：中断屏蔽字决定中断处理过程中能否被其他中断打断。
  - 中断优先级的两层含义
    - **中断响应优先级**：多个中断同时请求时，决定哪个先被响应。
    - **中断处理优先级**：在中断服务过程中，决定谁能打断谁。
    - **重要区别**：响应优先级由查询程序或硬件判优电路决定；处理优先级由各自的中断屏蔽字动态设定。
  - 多重中断处理流程
    - 1. **关中断**。
    - 2. **保存断点**。
    - 3. **中断服务程序寻址**。
    - 4. **保存现场和屏蔽字**。
    - 5. **开中断**。
    - 6. **执行中断服务程序**。
    - 7. **关中断**。
    - 8. **恢复现场和屏蔽字**。
    - 9. **开中断**。
    - 10. **中断返回**。
  - 多重中断的关键点
    - **隐指令阶段不能响应新中断**：从关中断到处理阶段开中断之前，CPU 不响应中断。
    - **屏蔽字起作用时机**：处理阶段开中断后，CPU 才检测多重中断，屏蔽字才开始起作用。
    - **打断条件**：只有第一个中断已经完成中断响应后，才可能被第二个中断打断。
- 11. 中断屏蔽字
  - 中断屏蔽的作用
    - **目的**：控制某个中断服务程序执行时，哪些中断可以打断它。
    - **紧急中断保护**：某些紧急中断不允许被新的中断打断。
    - **实现方式**：通过中断屏蔽字寄存器实现。
  - 屏蔽字含义
    - **0**：允许对应中断。
    - **1**：不允许对应中断，即屏蔽中断。
    - **处理优先级判断**：若某中断服务程序屏蔽了另一个中断，则后者不能打断前者。
  - 做题思路
    - 1. **先看响应优先级**：多个中断同时请求时，先按硬件排队或查询顺序确定响应顺序。
    - 2. **再看处理优先级**：进入服务程序后，根据屏蔽字判断能否被新中断打断。
    - 3. **注意返回主程序**：未被响应的中断通常要等 CPU 回到可响应状态后再响应。
    - 4. **画执行轨迹**：标出主程序、响应过程、各中断服务程序及被打断/恢复位置。
- 12. DMA 控制方式
  - DMA 基本概念
    - **DMA**：Direct Memory Access，直接存储器存取。
    - **基本思想**：用专门的 DMA 接口硬件控制外设与主存之间直接交换数据，不经过 CPU 逐字传送。
    - **DMA 控制器**：专门控制总线进行 DMA 传送的接口硬件。
    - **典型设备**：磁盘等高速设备。
  - DMA 与中断 I/O 的区别
    - **中断 I/O 请求的是处理器时间**：外设请求 CPU 执行服务程序。
    - **DMA 请求的是总线控制权**：DMA 控制器需要占用总线访问主存。
    - **传送粒度**：DMA 适合成批数据交换。
    - **CPU 参与度**：CPU 主要负责初始化和结束处理，传送过程基本交给 DMA 控制器。
  - DMA 传送方式
    - **挪用主存周期**：DMA 控制器挪用一个主存周期完成一次数据交换。
    - **独占若干主存周期**：DMA 控制器连续占用总线完成一批数据交换。
    - **请求-响应方式**：DMA 控制器通过请求和响应机制取得总线使用权。
- 13. 三种 DMA 方式
  - CPU 停止法
    - **做法**：DMA 传输时，DMA 控制器向 CPU 发停止信号。
    - **结果**：CPU 脱离总线，停止访问主存。
    - **持续时间**：直到 DMA 传送一块数据结束。
    - **特点**：DMA 效率高，但 CPU 被完全停住。
  - 周期挪用法
    - **做法**：CPU 让出一个总线事务周期。
    - **DMA 动作**：DMA 控制器挪用一个主存周期访问主存。
    - **传送粒度**：传送完一个数据后立即释放总线。
    - **特点**：单字传输方式，对 CPU 影响较小，是常见考点。
  - 交替分时访问法
    - **做法**：每个存储周期分成两个时间片。
    - **分配**：一个时间片给 CPU，另一个给 DMA 控制器。
    - **结果**：CPU 和 DMA 控制器在每个存储周期内都可访问存储器。
    - **特点**：规则清晰，但需要固定分时安排。
- 14. DMA 操作步骤
  - DMA 控制器中的寄存器
    - **主存地址寄存器**：保存内存缓冲区地址。
    - **字计数器**：保存需要传送的数据个数。
    - **设备地址寄存器**：保存外设地址。
    - **控制寄存器**：保存传送方向、启动命令等控制信息。
    - **其他控制逻辑**：控制设备通过总线与主存直接交换数据。
  - 第一步：DMA 初始化
    - **准备内存区**：
      - 输入数据时，申请并初始化内存缓冲区。
      - 输出数据时，在内存中准备好待输出数据。
    - **设置传送参数**：
      - 主存首地址。
      - 字计数值。
      - 传送方向。
      - 设备地址。
    - **启动 DMA**：CPU 发送“启动 DMA 传送”命令，然后可调度其他进程。
  - 第二步：DMA 数据传送
    - **CPU 不再参与逐字传送**：传送工作交给 DMA 控制器。
    - **DMA 给出内存地址**：由 DMA 控制器提供主存访问地址。
    - **读写线发命令**：控制读/写方向。
    - **数据线传数据**：完成外设与主存之间的数据交换。
    - **计数和改址**：每传送一个数据，字计数值减 1，并修改主存地址。
    - **结束条件**：字计数器为 0，表示所有 I/O 操作完成。
  - 第三步：DMA 结束处理
    - **发出中断请求**：DMA 完成后向 CPU 发出 DMA 结束中断请求。
    - **进入中断服务程序**：CPU 转去执行结束处理。
    - **后处理内容**：数据校验、状态检查、释放资源等。
- 15. DMA 与存储系统
  - DMA 带来的新访问路径
    - **无 DMA 时**：所有存储器访问都来自 CPU，并经过 MMU 地址转换和 Cache 层次。
    - **有 DMA 时**：DMA 设备直接访问主存，不经过 CPU 的地址转换机制和 Cache 层次。
    - **问题来源**：DMA 与虚拟存储、Cache 系统之间可能出现地址和一致性问题。
  - DMA 地址问题
    - **使用虚拟地址**：DMA 接口中必须有虚实地址转换机制。
    - **使用物理地址**：每次 DMA 传送通常不能跨页。
    - **原因**：连续虚页不一定映射到连续实页框。
    - **解决思路**：使用虚拟地址时，DMA 接口中可设置类似页表的地址映射表。
    - **初始化责任**：DMA 初始化时，由操作系统进行地址映射。
  - DMA 与 Cache 一致性问题
    - **问题**：DMA 直接访问主存，不经过 Cache。
    - **结果**：DMA 看到的主存值与 CPU 看到的 Cache 副本可能不同。
    - **本质**：主存和 Cache 之间出现数据不一致。
  - Cache 一致性解决方法
    - **方法 1**：让 I/O 活动通过 Cache 进行。
    - **方法 2**：由操作系统在 I/O 读时选择性使某些 Cache 块无效，在 I/O 写时迫使 Cache 写回。
    - **Cache 刷新**：写回或无效化相关 Cache 块的操作常称为 Cache 刷新。
    - **方法 3**：用硬件机制选择被刷新或无效的 Cache 项。
    - **适用场景**：硬件方式保证 Cache 一致性常用于多处理器系统。
- 16. 三种 I/O 控制方式对比
  - 横向对比

对比项	程序查询	中断控制	DMA 控制
CPU 参与	全程参与	响应和服务时参与	初始化和结束时参与
是否忙等待	是	否	否
数据路径	外设 -> CPU -> 主存	外设 -> CPU -> 主存	外设 <-> 主存
控制复杂度	最低	中等	较高
传送效率	低	中	高
适合设备	简单低速设备	低速/中速设备	高速批量设备

by joshuji 由 幕布 发布